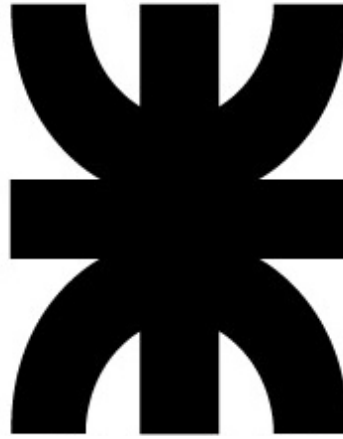


Universidad Tecnológica Nacional

Facultad Regional de Córdoba



Ingeniería en Sistemas de Información Ingeniería y calidad de software TP 6 - TDD - Justificación de diseño

Año 2025

Curso: 4K2

Grupo Nro: 13

Docentes:

- Ing. Meles Silvia Judith
- Ing. Massano María Cecilia

Integrantes:

- Lucas Nicolas Tripolone 95826
- Angel Rosales 95212
- Nicolas Olivera 95304
- Tomás Barrionuevo 95012
- Franco González 98913
- Julian Torres 94984
- Mauricio Herrera 95205
- Manuel Alzamora 95595
- Facundo Arana 96098
- Matias Farach 90594

Índice:

1. Descripción de la US	3
2. Arquitectura General	4
2.1 Separación Frontend/Backend	4
2.2 Patrones de Diseño	5
2.3 Seguridad	5
3. Testing	5
4. Convenciones de nombrado	7
4.1. Nombres de Archivos	7
4.2. Nombres de Clases	7
4.3. Nombres de Métodos y Funciones	7
4.4. Variables y Constantes	7
4.5. Rutas API (Endpoints)	7
4.6. Nombres de Carpetas	7

1. Descripción de la US

“Comprar entrada”

“COMO visitante QUIERO comprar una entrada PARA asegurar mi visita al parque”

Criterios de aceptación:

- Debe indicar la fecha de visita deseada, la cantidad de entradas requeridas, la edad de cada visitante y tipo de pase (VIP o regular).
- La fecha de visita guiada puede ser del día actual o futuro.
- Debe enviar un mensaje de confirmación vía mail.
- Debe redirigir a Mercado Pago al confirmar la compra si el pago es con tarjeta de crédito.
- La fecha de la visita debe estar dentro de los días en que el parque está abierto.
- Debe seleccionar la forma de pago: efectivo en caso de querer pagar en boletería o con tarjeta.
- La cantidad de entradas requeridas no debe ser mayor a 10.
- Al finalizar la compra se debe informar la cantidad de entradas compradas y la fecha.
- Se debe permitir la compra de entradas solo a usuarios registrados.

Pruebas de Usuario:

Probar comprar una entrada indicando la fecha de visita dentro de los días disponibles, una cantidad de entradas menor a 10, la edad de todos los visitantes, el tipo de pase, la forma de pago con tarjeta mediante Mercado pago y recepción del mail de confirmación. **[PASA]**

Probar comprar entradas ingresando una fecha de visita en la cual el parque se encuentra cerrado. **[FALLA]**

Probar comprar entradas sin seleccionar forma de pago. **[FALLA]**

Probar comprar entradas ingresando una cantidad de entradas mayor a 10. **[FALLA]**

2. Arquitectura General

El Trabajo Práctico se encuentra en el siguiente repositorio en la carpeta TrabajosPracticos/TP6_TDD

https://github.com/MatiasFarach13/ICSW_G13_4K2.git

2.1 Separación Frontend/Backend

- Frontend: Implementado en React con TypeScript.

Razones:

- TypeScript porque posee tipado estático que ayuda a prevenir errores, mejor soporte para IDE y autocompletado.
- React porque es una biblioteca madura y bien mantenida que cuenta con un gran ecosistema de componentes y facilidad para crear interfaces modulares.

- Backend: Desarrollado en Python usando Flask.

- Razones:

- Facilidad para TDD: Python tiene un excelente soporte para testing con pytest. Una sintaxis clara que facilita la escritura de tests legibles y herramientas maduras para mocking y fixtures.
- Framework Web: Flask por ser un framework ligero y flexible, ideal para APIs RESTful y de fácil integración con SQLAlchemy y otras librerías.
- Productividad, ya que cuenta con una sintaxis expresiva y clara con desarrollo rápido de prototipos y gran cantidad de bibliotecas disponibles.

- Librerías Backend Principales:

- SQLAlchemy: ORM maduro y bien documentado que abstrae la base de datos facilitando el testing y posee soporte para múltiples bases de datos.
- Flask-JWT-Extended: Para un manejo seguro de autenticaciones, integración con Flask y fácil implementación.
- Flask-CORS: Para la comunicación entre back y front con políticas de seguridad flexibles.

- Se optó por una arquitectura desacoplada para permitir el desarrollo independiente y facilitar el testing.

- Herramientas de Desarrollo

- Vite, ya que cuenta con construcción y desarrollo rápidos, soporte nativo para TypeScript y requiere configuración mínima.
- Pytest ya que es el framework de testing estándar en Python, posee una sintaxis expresiva para tests y tiene integración con cobertura de código.

- Base de Datos: SQLite
 - Sin necesidad de servidor separado. Perfecta para desarrollo y pruebas. Fácil respaldo y versionado. Portable y liviana
- Consideraciones de Seguridad
 - Werkzeug Security para manejo seguro de contraseñas, integración nativa con Flask e implementación probada de hashing.

Esta combinación de tecnologías proporciona un balance entre facilidad de desarrollo, mantenibilidad, y robustez, mientras mantiene el enfoque en las prácticas de TDD y la calidad del código.

2.2 Patrones de Diseño

- Patrón Gestor: Implementación del GestorCompraEntradas como punto central de la lógica de negocio.
- Patrón Repository: Uso de SQLAlchemy para abstraer la capa de datos.
- Patrón Strategy: Implementado en el manejo de diferentes tipos de entradas y cálculos de precios.

2.3 Seguridad

- Autenticación con implementación de JWT para manejo de sesiones, hash seguro de contraseñas y validación de correos electrónicos
- Validaciones de fechas y cantidades (no más de 10 entradas por compra, no se puede comprar para días lunes, 25/12 o 1/1, ni a mas de 30 días en el futuro)

3. Testing

test_compra_hoy_parcheado: Este test simula una compra para el día actual con diferentes tipos de entradas y edades (VIP y Regular, adultos, niños, seniors e infantes). Comprueba que el sistema calcula correctamente los precios con descuentos y permite compras válidas para el mismo día, verificando el cálculo del monto total considerando todas las categorías de edad.

test_compra_exitosa_con_fecha_valida_parcheada: Este test verifica una compra para un día futuro válido (no lunes ni feriado) con dos tipos de entradas diferentes. Comprueba que el sistema permite compras anticipadas y calcula correctamente los precios para una combinación simple de entradas VIP y Regular.

test_compra_con_tipos_combinados_y_varias_edades_parcheada: Este test prueba una compra con tres personas de diferentes edades (niño, adulto, senior) y diferentes tipos de entrada. Comprueba que el sistema aplica correctamente los descuentos por edad y maneja adecuadamente la combinación de diferentes tipos de entrada en una misma compra.

test_dos_compras_en_el_mismo_dia: Este test verifica que se pueden realizar múltiples compras en el mismo día hasta el límite máximo de entradas (10). Comprueba que no hay restricciones de cantidad total por día y que cada compra es independiente.

test_envio_email_confirmacion: Este test verifica que el sistema envía correctamente el email de confirmación después de una compra exitosa. Comprueba la integración con el sistema de notificaciones usando un mock del servicio de email.

test_compra_dia_anterior: Este test intenta realizar una compra para una fecha pasada. Comprueba que el sistema rechaza correctamente compras para fechas anteriores al día actual.

test_compra_mas_de_30_dias: Este test intenta realizar una compra con más de 30 días de anticipación. Comprueba que el sistema respeta el límite máximo de anticipación para las compras.

test_compra_para_feriado: Este test intenta realizar una compra para un día feriado (25 de diciembre). Comprueba que el sistema rechaza correctamente las compras en días donde el parque está cerrado por feriados.

test_compra_para_lunes: Este test intenta realizar una compra para un lunes. Comprueba que el sistema rechaza correctamente las compras para los días donde el parque está cerrado por política regular.

test_compra_mas_de_10_entradas: Este test intenta comprar más del límite permitido de entradas en una sola compra. Comprueba que el sistema aplica correctamente la restricción de cantidad máxima de entradas por transacción.

test_compra_email_invalido: Este test intenta realizar una compra con un formato de email inválido. Comprueba que el sistema valida correctamente el formato básico de las direcciones de email.

test_compra_email_vacio: Este test intenta realizar una compra con un email vacío. Comprueba que el sistema rechaza correctamente los campos de email vacíos.

test_compra_email_sin_arroba: Este test intenta realizar una compra con un email que no contiene el símbolo @. Comprueba que el sistema valida correctamente la presencia del carácter @ en las direcciones de email.

test_sin_forma_pago: Este test intenta realizar una compra sin especificar la forma de pago. Comprueba que el sistema requiere obligatoriamente una forma de pago válida para procesar la compra.

La suite de tests está bien estructurada siguiendo el principio de "Given-When-Then" y cubre:

1. Casos de uso exitosos
2. Validaciones de reglas de negocio
3. Manejo de errores
4. Validaciones de datos

5. Límites y restricciones del sistema

Además, utiliza mocks apropiadamente para aislar las pruebas de servicios externos (como el envío de emails) y para controlar el tiempo en pruebas que dependen de fechas.

4. Convenciones de nombrado

4.1. Nombres de Archivos

Python (Backend):

- Snake Case para archivos Python, por ejemplo `app.py`, `models.py`, `test_compraEntradas.py`.
- Prefijo `test_` para archivos de prueba por ejemplo, `test_models.py`, `test_compraEntradas.py`.
- Uso de `__init__.py` para marcar paquetes Python
- Clases en archivos separados con nombres descriptivos, por ejemplo `Compra.py`, `TipoEntrada.py`, `Usuario.py`

JavaScript/TypeScript/React (Frontend):

- Pascal Case para componentes React, por ejemplo `BaseLayout.jsx`, `ProtectedRoute.jsx`
- Camel Case para archivos utilitarios, por ejemplo, `counter.ts`, `main.tsx`
- Nombres descriptivos para páginas, por ejemplo `Comprar.jsx`, `ConfirmacionEfectivo.jsx`

4.2. Nombres de Clases

Python:

- PascalCase para nombres de clases, por ejemplo, `TipoEntrada`, `GestorCompraEntradas`, `User`, `Compra`.

Excepciones:

- Sufijo `Error` para clases de excepción, por ejemplo, `ParqueError`, `ParqueCerradoError`, `PagoInvalidoError`, `EmailInvalidoError`.

4.3. Nombres de Métodos y Funciones

Python:

- Snake Case para métodos y funciones, por ejemplo, `create_sqlite_db`, `get_session`, `validar_fecha`, `calcular_precio`, `set_precio_base`.
- Convenciones de Verbos

Uso de verbos descriptivos, por ejemplo, `validar_` para funciones de validación, `calcular_` para funciones de cálculo, `crear_` para funciones de creación, `get_` y `set_` para accesorios y mutadores.

4.4. Variables y Constantes

Python

- Snake Case para variables, por ejemplo, `fecha_visita`, `forma_pago`, `total_pagado`.
- `SCREAMING_SNAKE_CASE` para constantes, por ejemplo, `EDAD_INFANTE_MAX`, `EDAD_NINO_MAX`, `DIAS_ANTICIPACION_MAX`, `DESCUENTO_PORCENTAJE`.

4.5. Rutas API (Endpoints)

Uso de prefijo `/api/`, como, `/api/login`, `/api/register`.

4.6. Nombres de Carpetas

Nombres descriptivos en minúsculas, como frontend/, backend/, src/, test/.